Homework 3 – Findings

Question 1: Prompt Engineering Techniques

Zero-Shot Prompting

In order to test zero-shot prompting, a task was given to the model without any examples. Both a logical reasoning problem and a mathematical discount problem were effectively resolved by the model in the experiments. The answers were succinct and directly addressed the question. This demonstrated that when the task is simple and the instructions are sufficiently clear, zero-shot prompting is effective.

Few-Shot Prompting

Before presenting the actual problem, sample input-output pairs were used to assess few-shot prompting. The model was able to identify the pattern in the examples and apply it to new inputs, such as a sentiment classification task and a discount calculation problem. Few-shot prompting offered greater control over the anticipated response pattern and format than zero-shot prompting.

Chain-of-Thought Prompting

Chain-of-Thought prompting was employed to promote a methodical approach to problem solving. Both mathematical and logical reasoning tasks were part of the experiments. The answers made the solution simpler to understand by offering an explanation in between the final answer. Chain-of-Thought prompting yielded more thorough answers than zero-shot and few-shot prompting, and it was especially helpful for problems requiring several steps of reasoning.

Zero-Shot Chain-of-Thought Prompting

Without offering any examples of reasoning demonstrations, the Zero-Shot Chain-of-Thought prompting asked for a detailed solution. For both logical and mathematical reasoning problems, the model could still generate organized explanations. The experiment demonstrated that more explanatory answers can be obtained without the extra examples used in few-shot prompting when a straightforward instruction to reason step-by-step is added.

Meta-Prompting

In order to test meta-prompting, the model was asked to identify a suitable problem-solving strategy and then implement it. The model in the experiments distinguished between the

actual solution and the method of problem-solving. This resulted in a more organized response and showed that meta-prompting can be helpful when the approach taken to a problem is just as important as the solution.

Tree of Thoughts

Problems with several potential solutions were used to test the Tree of Thoughts prompting. The model produced a number of potential strategies in the scheduling and exam-preparation scenarios, assessed them, and then chose the best one. Because Tree of Thoughts explicitly considered alternatives rather than choosing a single solution right away, it generated the most comprehensive responses when compared to the other methods.

Q1 Overall Findings

All six prompting techniques effectively generated valuable responses, yet the organization and level of detail varied significantly. Zero-shot prompting yielded the most straightforward responses, whereas few-shot prompting assisted the model in adhering to a shown pattern. Chain-of-Thought and Zero-Shot Chain-of-Thought yielded clearer step-by-step justifications. Meta-prompting motivated the model to identify a strategy prior to addressing the task, while Tree of Thoughts examined various options before choosing a solution. As a result, the findings show that the selection of prompting methods can affect the organization and depth of an LLM's reply.

Question 2: Self-Attention and Causal Masking

Dataset and Implementation

The text data set was divided into words and transformed into numerical token IDs. An adjustable embedding layer was applied to depict the tokens as compact vectors. Self-attention with a single head and scaled dot-product was created from the ground up, involving distinct trainable Query, Key, and Value projections. The attention scores were calculated based on the Query and Key depictions, adjusted by the square root of the embedding size, and standardized through softmax. The resulting attention weights were used on the Value depictions. Training of the model centered on predicting the next token, ensuring that the embeddings and attention projections reflected learned parameters, not random starting points.

Unmasked Self-Attention Findings

The unmasked self-attention model allowed every position in the sequence to attend to every other position. After 500 training epochs, the model obtained a final training loss of **0.3436** and a next-token training accuracy of **80.00%**. The complete trained attention matrix had dimensions **51 × 51**, corresponding to the 51 tokens in the dataset.

Figure 1: Learned Unmasked Single-Head Self-Attention Matrix (executed in python file)

The attention heatmap without masks shows attention values that have been learned, appearing both above and below the main diagonal. This indicates that tokens can focus on positions both before and after their own in the sequence. Since future positions can be accessed, unmasked attention doesn't impose the limitation needed for autoregressive text generation.

Causal Masked Self-Attention Findings

For the second model, a lower-triangular causal mask was applied to the attention score matrix before the softmax operation. The mask prevents each token from attending to tokens that occur later in the sequence. The causal masked model was also trained for 500 epochs and obtained a final training loss of **0.1387** and a next-token training accuracy of **94.00%**.

Figure 2: Learned Causal Masked Single-Head Self-Attention Matrix(executed in python file)

The attention heatmap reveals the impact of masking by showing that attention values are concentrated on or below the main diagonal, with the upper-right part of the matrix showing no values. Through numerical analysis, it was confirmed that the highest attention weight given to any upcoming position was 0.0. This validates that the lower-triangular causal mask effectively blocked tokens from reaching future data.

Comparison of Unmasked and Masked Attention

The model without a mask had a final loss of 0.3436 and an 80.00% training accuracy, while the model with causal masking had a final loss of 0.1387 and a training accuracy of 94.00%. In this specific test, the masked model had lower training loss and higher training accuracy. However, this outcome is only relevant to the limited dataset and training setup and does not indicate that using causal masking always enhances prediction accuracy.

The key point to note is the difference in structure between the two attention mechanisms. Unmasked attention allows a token to consider both past and future positions, whereas causal attention limits a token to itself and past positions only. This restriction prevents future information from influencing the model and makes causal masking suitable for generating the next token in an autoregressive manner.

Conclusion

The tests showed how prompt layout influences responses from LLM and revealed the inner workings of self-focus. Various methods for designing prompts led to varying degrees of

direction, logical organization, and investigation. Through the self-focus trial, it was illustrated how the acquired representations of Query, Key, and Value influence attention among tokens. Moreover, the application of causal masking altered the open attention matrix into a triangular form, lowering attention to future tokens to 0.0, showing the process employed to maintain the autoregressive nature of generating the next token.